Game Integration Process

Game Server

1. Create Game Server logic

- If you are not using GDK module, you need to expose the REST API (see `GAME_SERVER_INTEGRATION.pdf`) and you should use RNG service (see `RNG_API.pdf`)
- If you are using GDK module, the API and RNG are handled under the hood, so you donâ€™t need to take care of that, and you just create game logic (see `GAME_SERVER_DEVELOPMENT.pdf`).

2. Publish Game Server

One or multiple games need to be wrapped into a Docker container. Once the container is built, you need to push it to our repository, so we can deploy it. Login to our private container registry (only once):

```
export TOKEN=XXX #XXX is Github token, request it from Tequity
echo $TOKEN | docker login ghcr.io -u user --password-stdin
```

and then build and push the container:

```
DOCKER_BUILDKIT=1 docker buildx build --platform linux/amd64 --push -t ghcr.io/slotify/games/CONTAINER_NAM
docker push ghcr.io/slotify/games/CONTAINER_NAME:v1.0.0
```

To update game container:

```
docker push ghcr.io/slotify/games/CONTAINER_NAME:v1.0.1 #increment the version
```

Each container can contain one or many games. Usually `CONTAINER_NAME` is the name of the provider but can be anything.

3. Deploy Game Server

Tequity may do this step depending on who owns/manages infrastructure.

In the main.tf file you add provider and the version you want to deploy

```
versions = {
  "games" = {
    "CONTAINER_NAME" = "v1.0.1"
  }
}
```

and run `terraform apply` to deploy the change.

4. Test Game Server

Once itâ€™s updated run Game Verifier tool in the Back Office and in case of errors troubleshoot based on error messages and logs.

Game Client

1. **Integrate Game Client**

   Your game framework should be integrated with a frontend library. There are a lot of optional methods, so you can focus initially only on the necessary ones. Details can be find in `CONNECTOR_GAME_CLIENT_API.pdf`. When running connector from a local machine, you can use an absolute path to a `connector` library i.e.

```
https://cdn-test.tequity.ventures/slotify/connector/connector.js
```

But once it is uploaded, it should use a relative path

```
/slotify/connector/connector.js
```

so it uses the library deployed in the same environment as a game, and the same game client bundle can be deployed across multiple environments.

2. **Deploy Game Client**

   Tequity may do this step depending on who owns/manages infrastructure.

   Once integrated, we can deploy files to our CDN or you can launch it from yours.

```
gsutil -m rsync -r MY-GAME gs://cdn-GCP-PROJECT/MY_PROVIDER/MY_GAME
```

By default, all static files have quite aggressive caching so if you want to remove some files from being cached (i.e. `index.html`) you need to run:

```
gsutil setmeta -h "Cache-Control: max-age=0, no-cache" gs://cdn-GCP-PROJECT/MY_PROVIDER/MY_GAME/index.html
```